

CSC 3114: Cryptology and Coding Theory

Take-Home Assignment (Test 2)

Group size:	Maximum 4 students	Due date:	7 October 2026, 11:59 PM EAT
Submission:	One PDF per group	Video:	Link included in PDF; maximum 5 mins
Presentation:	All members must participate	Coding:	Python Programming Language

Overview

This assignment combines theory and practical work from Block Ciphers, AES, Integrity and Stream Ciphers. Your group will investigate confusion, diffusion, and avalanche behaviour; use AES in practice; compare modes of operation; and demonstrate why integrity and authentication are necessary. Use standard cryptographic libraries for AES rather than implementing a production cipher from scratch.

Task 1: Theory and Cipher Design

[10 marks]

Answer concisely and in your own words.

- Explain **confusion**, **diffusion** and the **avalanche effect**. Relate each concept to DES or AES. [4]
- Compare DES and AES using: block size, key size, number of rounds, cipher structure, and present-day security suitability. [3]
- Explain the main security differences among **ECB**, **CBC** and **CTR**. State the purpose of an IV/nonce and identify one important misuse to avoid. [3]

Task 2: Python Avalanche Experiment

[10 marks]

Use AES-128 to encrypt one 16-byte plaintext block. Flip exactly one bit in the plaintext, encrypt again with the same key, and count how many ciphertext bits change. Repeat for at least **10 different single-bit flips**. Present the results in a small table and compute the average percentage of changed ciphertext bits. Briefly interpret the results.

You may use/adapt the following guiding functions:

```
def flip_bit(data: bytes, bit_index: int) -> bytes:
    b = bytearray(data)
    byte_i, offset = divmod(bit_index, 8)
    b[byte_i] ^= (1 << offset)
    return bytes(b)

def count_changed_bits(x: bytes, y: bytes) -> int:
    return sum((a ^ b).bit_count() for a, b in zip(x, y))

def avalanche_trial(encrypt_block, plaintext: bytes, bit_index: int):
    c1 = encrypt_block(plaintext)
    c2 = encrypt_block(flip_bit(plaintext, bit_index))
    return count_changed_bits(c1, c2)
```

Task 3: AES Modes in Practice

[10 marks]

Choose a message of at least 40 characters. Using a standard Python cryptographic library:

- encrypt and decrypt the message using **AES-CBC** with a fresh IV and correct padding;
- encrypt and decrypt the same message using **AES-CTR** with a fresh nonce/counter;
- verify that the recovered plaintext exactly matches the original;
- report the IV/nonce in hexadecimal, ciphertext length, and two observations comparing CBC and CTR.

Guiding function skeletons:

```
def aes_cbc_encrypt(key: bytes, plaintext: bytes):
    # Generate fresh IV, pad, encrypt, return (iv, ciphertext)
    pass

def aes_cbc_decrypt(key: bytes, iv: bytes, ciphertext: bytes):
    # Decrypt, remove padding, return plaintext
    pass

def aes_ctr_roundtrip(key: bytes, plaintext: bytes):
    # Use fresh nonce/counter, encrypt, decrypt, return evidence
    pass
```

Task 4: Integrity and Tampering

[6 marks]

Compute both a **SHA-256 hash** and an **HMAC-SHA256** tag for your message. Change one character in the message and recompute both. Demonstrate:

- how the digest/tag changes after tampering;
- successful HMAC verification for the original message;
- failed verification for the modified message;
- why a plain hash does **not** authenticate the sender, while HMAC provides integrity and authentication using a secret key.

```
import hashlib, hmac

def sha256_digest(message: bytes) -> str:
    return hashlib.sha256(message).hexdigest()

def make_hmac(key: bytes, message: bytes) -> bytes:
    return hmac.new(key, message, hashlib.sha256).digest()

def verify_hmac(key: bytes, message: bytes, tag: bytes) -> bool:
    expected = make_hmac(key, message)
    return hmac.compare_digest(expected, tag)
```

Task 5: Reflection and Video Presentation

[4 marks]

Write a short reflection of not more than **200 words** stating the most important security lesson your group learned from the experiments.

Record a **maximum 5-minute video presentation**. The presentation must:

- include **all group members**;
- show evidence that the code runs;
- explain the security meaning of the results, rather than reading code line by line;
- be uploaded to a platform accessible to the lecturer, with the **working link included in the submitted PDF**.